

Week 11

1. Named Pipes (II): FIFO

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Monday, November 10th, 2025



University of Windsor

Content

Midterm Exam

Brief review – Statistics

FIFOs or Named Pipes

The `mkfifo()` library function

Examples

Part I

Midterm Exam: Review



Midterm Exam: Review

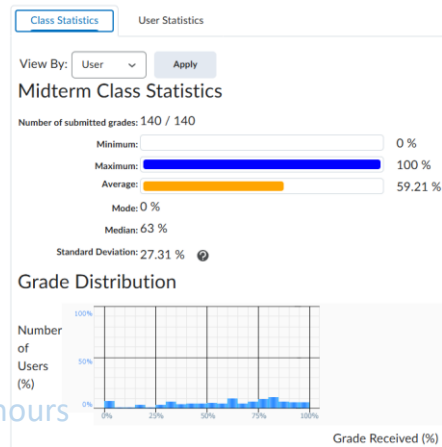
- Format issues:
 - Student information missing / not added correctly.
 - Difficult to read writing
 - Messy writing
- Understanding issues: Questions / exercises on coding
 - What you should do:
 - Read carefully
 - Fully understand
 - Ask for clarifications
 - Prioritize
 - Structure and design
 - Code
 - Review and submit
- Review Midterm Exam papers: During office hours



Midterm Exam: Statistics

- Format issues:
 - Student information missing / not added correctly.
 - Difficult to read writing
 - Messy writing
- Questions:
 - Questions / exercises on coding
 - Read carefully
 - Fully understand
 - Ask for clarifications
 - Prioritize
 - Structure and design
 - Code
 - Review and submit
- Review Midterm Exam papers: During office hours

Item Statistics: Midterm



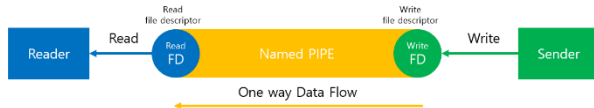
Part II

Named Pipes



Named Pipes (FIFO)

- Named Pipes (FIFOs):
 - Are a type of inter-process communication (IPC).
 - Allows unrelated programs to exchange data through a special file in the filesystem.
 - They provide a **named**, one-way or duplex channel for processes to write and read data, and
 - Can be used for communication on a single machine or across a network.



Named Pipes (FIFO): Key characteristics

- **File System Representation:**
Named pipes are created as special files in the file system using commands like `mkfifo` or system calls like `mkfifo()`. They can be viewed and managed like other files, for example, with the `ls` command.
- **Inter-Process Communication (IPC):**
They provide a mechanism for different processes to exchange data. One or more processes can write data to the named pipe, and another process can read from it.
- **First-In, First-Out (FIFO) Behavior:**
Data written to a named pipe is read in the same order it was written, adhering to the FIFO principle.
- **Unidirectional (typically):**
While some systems might support bidirectional anonymous pipes, named pipes are strictly unidirectional, meaning data flows in one direction (from writer to reader).
- **Persistence:**
Unlike unnamed pipes that disappear when the processes using them terminate, named pipes persist in the file system until explicitly removed (e.g., using `rm`).
- **Blocking Behavior:**
Opening a named pipe for reading or writing typically blocks until the other end (writing or reading, respectively) is also opened, ensuring both sides are ready for communication. Non-blocking modes are also possible.



Named Pipes (FIFO): How they work

- A process creates a named pipe (using `mkfifo` or `mkfifo()`), specifying a path in the file system.
- Processes wanting to communicate open this named pipe file using standard file operations like `open()`. One process opens it for writing, and another opens it for reading.
- Once opened by both ends, data can be written to the pipe using `write()` and read from it using `read()`, just like with a regular file.
- The kernel handles the data transfer internally, without writing the data to the physical disk, making it an efficient IPC mechanism.



Named Pipes (FIFO) vs Unnamed Pipes: A comparison

PIPE	NAMED PIPE (FIFO)
Communications between related processes	Communications between both related and unrelated processes
Does not persist beyond the lifetime of the processes	Persists beyond the lifetime of the processes
Not visible in the file system	Visible in the file system
Process access only	File access permissions can be set
<code>pipe()</code> system call	<code>mkinfo</code> command, <code>mkfifo()</code> , or <code>mknod()</code> system call
Identified by file descriptors	Identified by a name, or a pathname



Named pipes

FIFOs or Named Pipes

FIFOs (First In First Out), also called *Named Pipes*, offer the following advantages over pipes:

- They have a name that exists in the **file system**.
They can be used by **unrelated processes**.
- They exist until explicitly **deleted**.

The `mkfifo()` library function

`mkfifo()`

Synopsis:

```
int mkfifo(const char *path, mode_t mode)
```

`mkfifo()` returns 0 if OK, -1 otherwise.

Creating a FIFO is similar to creating a file.

Example: `mkfifo("/tmp/channel.fif", 0755);`

Note

`mkfifo()` is actually a library function (`#include <sys/stat.h>`) that invokes the system call `mknod()`, used to create special files.

FIFO manipulation

Once a FIFO has been created, it can be treated as a file. In particular, the system calls `open()`, `close()`, `read()`, `write()` and `unlink()` (to delete a file) can be used on a FIFO.

- To **create** a FIFO: Use `mkfifo` library function.
- To **remove** a FIFO: Use `unlink` system call:
`unlink(fifoname)`
- To **read from a FIFO**: Use `open(fifoname, O_RDONLY)`, then use `read` system call.
- To **write to a FIFO**: Use `open(fifoname, O_WRONLY)`, then use `write` system call.

FIFO manipulation

By default, we have:

- Calling `open()` for read-only blocks the caller until some other process opens the FIFO for writing.
- Calling `open()` for write-only blocks the caller until some other process opens the FIFO for reading.
- If a process writes to a FIFO that no process has opened for reading, the signal **SIGPIPE** will be generated.
- When the last writer for a FIFO closes the FIFO, an **end-of-file** will be generated for the reader.
- Like pipes, FIFOs are one-way communication channels.

Note

In case of multiple processes writing to the same pipe, a `write` of up to `PIPE_BUF` bytes is guaranteed to be atomic.

Part III

Named Pipes (Examples)



○ Client/Server Architecture



Client/Server Architecture

- **Client:**

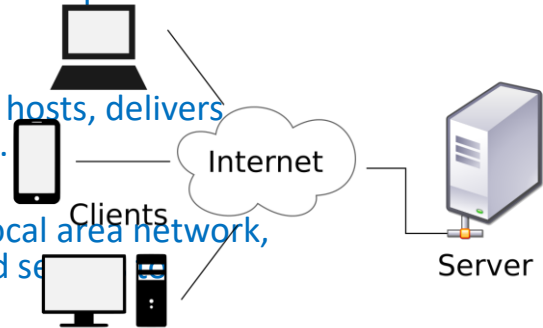
A device or application (e.g., web browser, mobile app, desktop application) that initiates requests for resources or services.

- **Server:**

A powerful computer or system that hosts, delivers and manages resources and services.

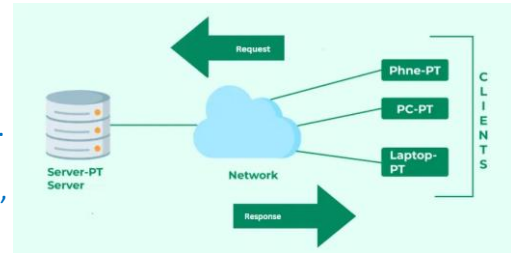
- **Network:**

The communication medium (e.g., local area network, the internet) that enables clients and servers to interact and exchange data.



Client/Server Request-Response Model

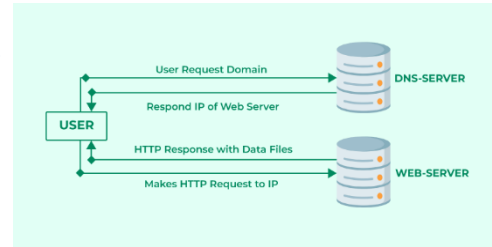
- **Client Request:**
A client sends a request for a specific service or resource to the server over the network.
- **Server Processing:**
The server receives the request, processes it, and performs any necessary actions, such as retrieving data from a database or executing application logic.
- **Server Response:**
The server then sends a response back to the client, containing the requested data or the result of the performed action.
- **Client Processing:**
The client receives the response and processes it, presenting the information to the user or performing further actions based on the server's response.



Client/Server Request-Response Example

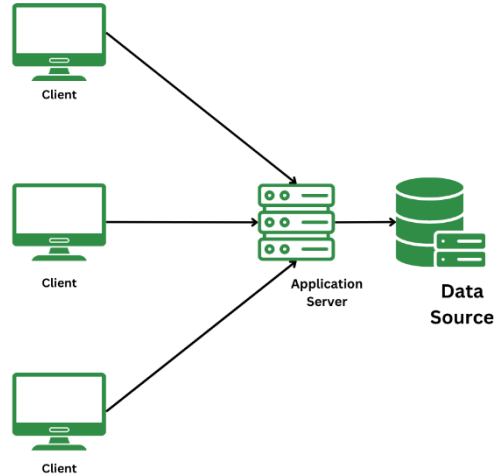
How a Web Browser interacts with the Servers?

- The process of interacting with servers through a browser involves several steps:
 - a. User Enters the URL (Uniform Resource Locator):
The user types a website address (e.g., www.example.com) into the browser's address bar.
 - b. DNS (Domain Name System) Lookup:
The browser contacts a DNS server to convert the domain into an IP address.
 - c. Establishing a Connection:
The browser sends an HTTP/HTTPS request to the server using the resolved IP address.
 - d. Server Responds:
The server sends back website files (HTML, CSS, JavaScript, images).
 - e. Browser Renders the Webpage:
 - a. DOM interpreter: Processes HTML to structure the page.
 - b. CSS interpreter: Applies styles
 - c. JavaScript Engine: Adds interactivity (using JIT compilation for performance).



Client/Server Three-Tier Architecture

- Logical/Physical Tiers:
 - Presentation (Client)
 - Application (Business Logic)
 - Data (Database)
- Advantages:
 - Scalability
 - Modularity and Maintainability
 - Flexibility
 - Security



○ Example 1: Client/Server



Example 1: A client/server application

The goal of the following example (Example 1) is to write a client/server application where a server accepts data from clients using the FIFO `~/myserver`.

○ server1.c



Example 1: A client/server application...(server part)

```
#include <wait.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/stat.h>
#include <stdlib.h>
#include <strings.h>
#include <string.h>

void child(pid_t client);

int main(int argc, char *argv[]){
    int fd, status;
    char ch;
    pid_t pid;

    unlink("/tmp/sc2");

    if(mkfifo("/tmp/sc2", 0777)){
        perror("main");
        exit(1);
    }

    while(1){
        fprintf(stderr, "Waiting for a client\n");

        fd = open("/tmp/sc2", O_RDONLY);
        fprintf(stderr, "Got a client: ");
        read(fd, &pid, sizeof(pid_t));
        close(fd);

        fprintf(stderr, "%ld\n", pid);

        if(fork()==0)
            child(pid);
        else {
            waitpid(0, &status, WNOHANG);
            sleep(3);
            //printf("child pid=%d terminated.\n", wait(NULL));
        }
    }
}
```

Example 5: A client/server application...(server part)

```
void child(pid_t pid){
    char fifoName[100];
    char newline = '\n';
    int fd, i;

    sprintf(fifoName, "/tmp/fifo%d", pid);

    unlink(fifoName);

    mkfifo(fifoName, 0777);

    fd = open(fifoName, O_WRONLY);

    for(i=0; i < 10; i++){
        write(fd, fifoName, strlen(fifoName));
        write(fd, &newline, 1);
        sleep(1);
    }
    close(fd);
    unlink(fifoName);
    exit(0);
}
```

○ client1.c



Example 5: A client/server application...(client.c)

```
//client.c
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/stat.h>

int main(int argc, char *argv[]){
    int fd;
    char ch;

    while((fd=open("/tmp/server", O_WRONLY))===-1){
        fprintf(stderr, "trying to connect\n");
        sleep(1);
    }

    printf("Connected: type in data to be sent\n");

    while((ch=getchar()) != -1) // -1 is CTRL-D
        write(fd, &ch, 1);
    close(fd);
}
```

Part IV

Homework



Homework

- Modify the Client/Server example:
 - 2-way communication between Client and Server
 - Multi-client implementation (many clients contact the server)
 - Bi-directional communication in the multi-client communication



Part V
Attendance Quiz (Not graded)



Quiz



Attendance Quiz

